



UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia (TUPaD)

Trabajo Final Integrador (TFI)

Propuesta de Proyecto — 1.ª Entrega

NexoVet

Sistema Integral de Gestión Veterinaria



Integrantes

Angelelli, Rodrigo Martín · Schneider, Astrid Elizabeth

Tutora

María Candela Grosso

Repositorio GitHub

<https://github.com/roangelelli/NexoVet>

Fecha de entrega

26 / 08 / 2026 (fecha máxima: 30/08)

1. Definición del problema

Muchas veterinarias de pequeña y mediana escala gestionan su operación diaria de forma manual o con herramientas dispersas: agendas de papel, planillas de cálculo y archivos físicos para las historias clínicas. Esta modalidad genera turnos superpuestos, información difícil de recuperar, historias clínicas incompletas y pérdida de datos ante extravíos o cambios de personal.

Del lado del cliente (el dueño o dueña de la mascota) el problema es igual de concreto: para solicitar un turno debe llamar por teléfono en horario de atención, no tiene forma de consultar el historial médico de su mascota ni de saber cuándo corresponde la próxima vacuna. Toda esa información queda dentro de la veterinaria y resulta inaccesible para quien más la necesita.

Se trata, por lo tanto, de una problemática local y real, presente en cualquier barrio o localidad, con impacto directo en la calidad de atención y en la organización del establecimiento.

2. Objetivo del proyecto

Objetivo general

Desarrollar una aplicación web responsiva que centralice la gestión de una veterinaria y, a la vez, ofrezca a los clientes un canal de autogestión para solicitar turnos y consultar la información médica de sus mascotas.

Objetivos específicos

- Digitalizar la administración de turnos, clientes, mascotas e historias clínicas.
- Permitir que cada cliente registre y seleccione sus mascotas, contemplando que un mismo cliente puede tener más de una.
- Permitir que el personal configure la disponibilidad de atención, a partir de la cual el sistema genera automáticamente las franjas de turnos.
- Registrar cada atención con diagnóstico, tratamiento, vacunas e imágenes adjuntas (por ejemplo radiografías o fotografías clínicas).
- Ofrecer al cliente una vista propia para solicitar y cancelar turnos, ver la historia clínica de sus mascotas y el carnet de vacunas.
- Exponer una API única, consumida por ambos perfiles de usuario, sobre una misma base de datos.
- Desplegar el sistema en servicios en la nube, accesible desde cualquier navegador, incluido el del teléfono.

3. Solución propuesta

Se propone una única aplicación web responsiva construida sobre una arquitectura de tres capas: una interfaz de usuario (frontend), una API que concentra la lógica de negocio (backend) y una base de datos relacional. La misma aplicación presenta dos experiencias diferenciadas según el rol de quien inicia sesión:

- **Perfil interno (veterinaria):** el personal gestiona la disponibilidad de atención, los turnos, los clientes, las mascotas, las historias clínicas y las vacunas.
- **Perfil cliente (dueño/a):** registra y selecciona sus mascotas, solicita y cancela turnos, y consulta la historia clínica y el carnet de vacunas.

Al ser una webapp responsiva, el cliente accede desde el navegador de su celular a través de una URL, sin necesidad de instalar ninguna aplicación. Esto cubre el requerimiento de acceso móvil sin el costo de desarrollar una aplicación nativa por separado.

4. Alcance

Funcionalidades incluidas

- Autenticación de usuarios y control de acceso por roles (personal / cliente).
- Gestión de clientes y de sus mascotas; un cliente puede registrar y seleccionar varias mascotas.
- Configuración de la disponibilidad de atención por parte del personal, con generación automática de franjas de turno.
- Solicitud, confirmación, atención y cancelación de turnos, con manejo de estados.
- Registro y consulta de la historia clínica, con imágenes adjuntas a cada atención.
- Gestión del carnet de vacunas con cálculo de la próxima fecha de aplicación.
- Reportes simples de uso interno (por ejemplo, turnos del día).

Fuera del alcance (posibles mejoras futuras)

Con el fin de garantizar la viabilidad dentro de los plazos, se excluyen deliberadamente de esta primera versión: pagos en línea, notificaciones push, mensajería o chat interno, asistentes basados en inteligencia artificial, integración con controladores fiscales y el desarrollo de una aplicación móvil nativa. Estas quedan documentadas como posibles extensiones a mencionar en la defensa.

5. Módulos del sistema

El proyecto se organiza en los siguientes módulos funcionales, que servirán de base para el listado necesario para la 2.ª entrega:

Módulo	Descripción
Usuarios y roles	Registro, inicio de sesión y permisos diferenciados por rol (personal / cliente).
Clientes	Alta y administración de los dueños de las mascotas.
Mascotas	Alta, edición y selección de mascotas; un cliente puede tener varias asociadas.
Disponibilidad	Configuración de días y horarios de atención por parte del personal.
Turnos	Ciclo de vida del turno (solicitado → confirmado → atendido / cancelado); franjas calculadas según la disponibilidad.
Historia clínica	Registro de cada atención vinculada a una mascota, con imágenes adjuntas.
Vacunas	Aplicaciones registradas y cálculo de la próxima fecha a vacunar.
Reportes	Consultas de apoyo a la gestión diaria.

6. Prototipo de interfaz (bocetos)

A continuación se presentan bocetos preliminares de las pantallas principales, elaborados para visualizar cómo se vería el producto. Se trata de representaciones orientativas; el diseño definitivo podrá ajustarse durante el desarrollo.

Vista del cliente (webapp responsiva en el celular)



Inicio de sesión (acceso unificado)



Mis mascotas: alta y selección

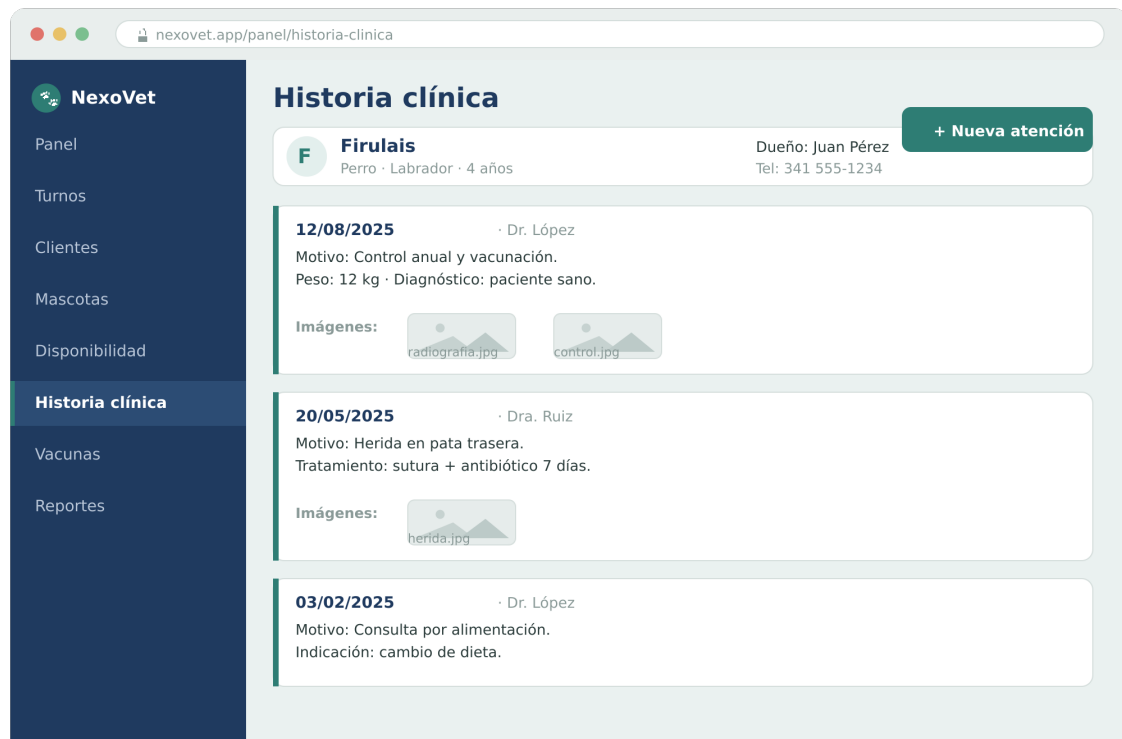


Solicitud de turno con franjas libres

Vista del personal (panel de gestión web)



Configuración de disponibilidad: define los horarios que generan las franjas de turno.



Historia clínica con imágenes adjuntas a cada atención (radiografías, fotografías).

7. Stack tecnológico

La totalidad del proyecto se desarrolla sobre JavaScript, lo que reduce la complejidad al mantener un mismo lenguaje en todas las capas y facilita su deploy.

Componente	Tecnología	Justificación
Lenguaje	JavaScript	Único lenguaje de punta a punta; simplifica el desarrollo y la comprensión.
Frontend	React.js	Interfaz responsiva basada en componentes; se adapta a escritorio y celular.
Backend	Node.js + Express	API REST que centraliza la lógica de negocio y el acceso a datos.
Base de datos	MySQL (relacional) — a confirmar	Modelo relacional claro; proveedor de hosting a definir.
Almacenamiento de imágenes	A confirmar	Servicio dedicado para las imágenes de la historia clínica.
Despliegue / Plataforma	A confirmar	Servicios en la nube que cumplen el requisito de componentes online.
Control de versiones	Git + GitHub	Repositorio único donde vive todo el proyecto.

Nota sobre las imágenes: las imágenes de la historia clínica no se almacenarán dentro de la base de datos, sino en un servicio de almacenamiento externo, guardando en la base únicamente la URL de cada archivo. Esto mantiene la base de datos liviana como práctica recomendada.

8. Plan de trabajo y cronograma

El trabajo se organiza siguiendo las instancias de entrega definidas por la cátedra:

Etapa	Entregable / Actividad	Fecha
1.ª Entrega	Propuesta de proyecto, stack tecnológico y declaración del repositorio único de GitHub.	30/08
2.ª Entrega	Esquema de la base de datos y listado de módulos aprobados. Otorga la condición de Regular.	27/09
Desarrollo	Construcción iterativa de los módulos, pruebas y despliegue en la nube.	28/09 – 13/11
Entrega Final	Repositorio completo, despliegue funcionando, informe escrito y video explicativo (preferentemente en inglés).	14/11
Defensa Oral	Presentación y justificación del proyecto ante el comité en mesa de examen.	Mesa de examen

9. Repositorio único en GitHub

Todo el proyecto se aloja y desarrolla dentro de un único repositorio de GitHub, que se mantendrá estructurado, limpio y profesional. Su organización inicial prevista es:

- **README.md:** documentación del proyecto, instrucciones de instalación, tecnologías e integrantes.
- **/frontend:** código fuente de la interfaz en React.
- **/backend:** código fuente de la API en Node.js.
- **/database:** scripts DDL/DML, esquema y migraciones.
- **/docs:** informes, esquemas de avance y documentación de las entregas.

URL del repositorio: <https://github.com/roangelelli/NexoVet>